Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти

1. Характеристики системы

Процессор: Intel Core i5-12450H, 8 ядер (4 производительных P-ядра и 4 энергоэффективных E-ядра), 12 логических потоков. Кеш L1 данных: 48 КБ на P-ядро, 32 КБ на E-ядро; L2: 1,25 МБ на P-ядро и 2 МБ на группу из 4 E-ядер; L3: 12 МБ, общий. Компилятор: GCC 13.3.0, WSL Ubuntu 24.04, ключи -O2 -fopenmp.

2. Результаты экспериментов

Таблица 1. Эксперимент 1. Влияние порядка обхода

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0.000165	0.000044	6.35	23.67
512	0.000854	0.000566	4.91	7.41
1024	0.004045	0.005019	4.15	3.34
2048	0.061252	0.049182	1.10	1.36
4500	0.109335	0.133351	2.96	2.43

Таблица 2. Эксперимент 2. Влияние размера блока

Размер блока b	Время, с	BW, ГБ/с
4	0.064297	5.04
8	0.046422	6.98
16	0.033609	9.64
32	0.031027	10.44
64	0.035270	9.19
128	0.035761	9.06

Таблица 3. Эксперимент 3. Влияние количества потоков (при b* = 32)

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0.121674	0.90	0.034018	3.21
2	0.056371	1.94	0.021926	4.99
4	0.030654	3.57	0.014526	7.53
8	0.022927	4.77	0.013627	8.02
16	0.021368	5.12	0.013158	8.31

Таблица 4. Эксперимент 4. False sharing

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0.027454	11.80
schedule(static, 1)	0.029814	10.87
schedule(static, 8)	0.026690	12.14

По результатам дополнительного 5 эксперимента параллельная in-place версия при 8 потоках выполнилась за 0.0090 с, то есть в 1.5 раза быстрее transpose_blocked_omp (0.0136 с).

3. Вывод

В первом эксперименте rows и cols работали с разной скоростью, потому что в каждой версии одна из матриц обходится с шагом в целую строку: в rows так записывается B, а в cols читается A. Пока матрицы маленькие (N = 256 и 512) и помещаются в кеш, это почти не мешает, но начиная с N = 1024 они уже не помещаются в L3, и пропускная способность падает до 1-3 ГБ/с. Во втором эксперименте лучше всего оказался блок b = 32: два таких блока (из A и из B) занимают 16 КБ и целиком помещаются в кеш L1, поэтому загруженные данные используются полностью. Блоки 4 и 8 слишком маленькие, а блоки 64 и 128 уже не помещаются в L1, поэтому с ними программа работала медленнее. Ускорение здесь ограничено сильнее, чем в ЛР2, потому что при транспонировании программа только читает и записывает память, а память у всех потоков общая. Поэтому после 4 потоков пропускная способность почти не растёт, и переход с 4 на 16 потоков ускорил blocked_omp всего на 10%, а в ЛР2 - на 43%. В четвёртом эксперименте schedule(static, 1) оказался медленнее schedule(static) примерно на 9% из-за false sharing: соседние строки A достаются разным потокам, они пишут в одну и ту же кеш-линию B, и она постоянно передаётся между ядрами. При static и static, 8 каждый поток берёт несколько строк подряд, и такой проблемы почти нет.